Software Requirements Specification

Personal Diary System

[Team ID] – Personal Diary

Supervisor TA: Akram Tarek

Team Members

Name (Arabic)	Seat No.	Dept.
مصطفى محمد حمدي محمد	2023170607	CSYS
محمد عمر احمد مختار حرش	2023170524	CSYS
محمود شريف محمد محمود	2023170558	SC
محمد جميل خلف احمد	2023170488	IS
علي يحيى محمد جابر الصادق	2023170374	CSYS
محمد عبدالغفار عباس السبكي	2023170514	SC
يحيى احمد يحيى فايز	2023170704	CSYS
سيف سيد محمد النواوي	2023170286	CSYS

1. Introduction

The Personal Diary System is a C# console application that allows individuals to record, search, and manage their daily diary entries. All data is stored persistently in a SQL Server database and accessed through ADO.NET using raw SQL queries written in C#.

The primary purpose of this project is to demonstrate real-world database interaction using C# and ADO.NET: establishing SqlConnection objects, executing parameterized SqlCommand queries, reading results with SqlDataReader, and handling database errors properly.

The system is operated entirely through a text-based console menu on a Windows machine with a local SQL Server instance.

2. User Requirements

The following requirements are expressed from the user's perspective:

- As a user, I want to log in with username and password so only I can access my diary.
- As a user, I want to write a new diary entry (title, date, body, mood) and have it saved to the database.
- As a user, I want to view all my diary entries listed by date.
- As a user, I want to search my entries by keyword or date range to quickly find a specific one.
- As a user, I want to see a monthly mood summary showing how many entries I wrote per mood.
- As a user, I want to update or delete an existing diary entry.

3. Functional Requirements

The following core functional requirements define what the system must accomplish.

FR-01: User Authentication

Description	The system checks username and bcrypt-hashed password against the Users table in SQL Server via ADO.NET.
Inputs	Username (string), Password (string)
Source	User (typed into the console)
Pre-condition	The Users table exists. The user has a registered account.
Post-condition	A session variable stores the authenticated userId for the console session.
Output	"Login successful. Welcome, [username]." OR "Invalid credentials. Try again."

FR-02: Create Diary Entry

Description	Inserts a new diary entry into DiaryEntries using a parameterized INSERT command.
Inputs	Title (string), Body text (string), Mood (Happy/Sad/Neutral/Anxious), Date (defaults to today)
Source	User (typed into console prompts)
Pre-condition	User is authenticated. The DiaryEntries table exists.
Post-condition	A new row is inserted into DiaryEntries with the correct UserId foreign key.
Output	"Entry saved successfully. Entry ID: [id]"

FR-03: View All Entries

Description	Retrieves and displays all diary entries for the logged-in user, ordered by date descending, using SqlDataReader.
Inputs	None (uses current session userId)
Source	User selects "View All Entries" from the main menu
Pre-condition	User is authenticated.
Post-condition	All matching rows are read from DiaryEntries and printed to the console.
Output	Numbered list: ID | Date | Title | Mood. OR "No entries found."

FR-04: Search Diary Entries

Description	Filters entries by keyword (in title and body) or date range using a parameterized SELECT query.
Inputs	Keyword (string, optional) and/or Start date / End date (optional)
Source	User selects "Search Entries" and enters criteria
Pre-condition	User is authenticated.
Post-condition	The SQL WHERE clause filters entries matching the criteria.
Output	List of matching entries OR "No entries matched your search."

FR-05: Mood Summary

Description	Queries the database with GROUP BY to count entries per mood for the current month.
Inputs	None (uses current userId and current month)
Source	User selects "Mood Summary" from the menu
Pre-condition	User is authenticated.
Post-condition	A GROUP BY aggregation runs on DiaryEntries.
Output	"Happy: 5 | Sad: 2 | Neutral: 8 | Anxious: 1"

4. Non-Functional Requirements

#	Category	Description
1	**Security**	Passwords stored as bcrypt hashes — never plain text. All SQL queries must use parameterized SqlCommand. String concatenation in SQL is strictly forbidden (SQL injection prevention).
2	**Performance**	Any single database query should return results within 2 seconds on a local SQL Server instance.
3	**Usability**	The console menu must be numbered and self-explanatory. A first-time user should navigate without a manual.
4	**Reliability**	All database operations wrapped in try/catch(SqlException). Meaningful error messages printed; application must not crash on bad input.
5	**Portability**	Runs on Windows 10+ with .NET 6+ and SQL Server (LocalDB accepted for development).
6	**Dev Constraints**	Language: C# (.NET 6+). Database: SQL Server / LocalDB. Data access: ADO.NET only — no Entity Framework. Version control: Git with feature branches.
7	**Maintainability**	All database logic isolated in a separate DatabaseHelper / Repository class. No raw SQL queries inside Program.cs.

5. Use Case Diagram

The diagram below shows the two actors (User and Admin) and the five core use cases. Lines represent associations between actors and the use cases they participate in.

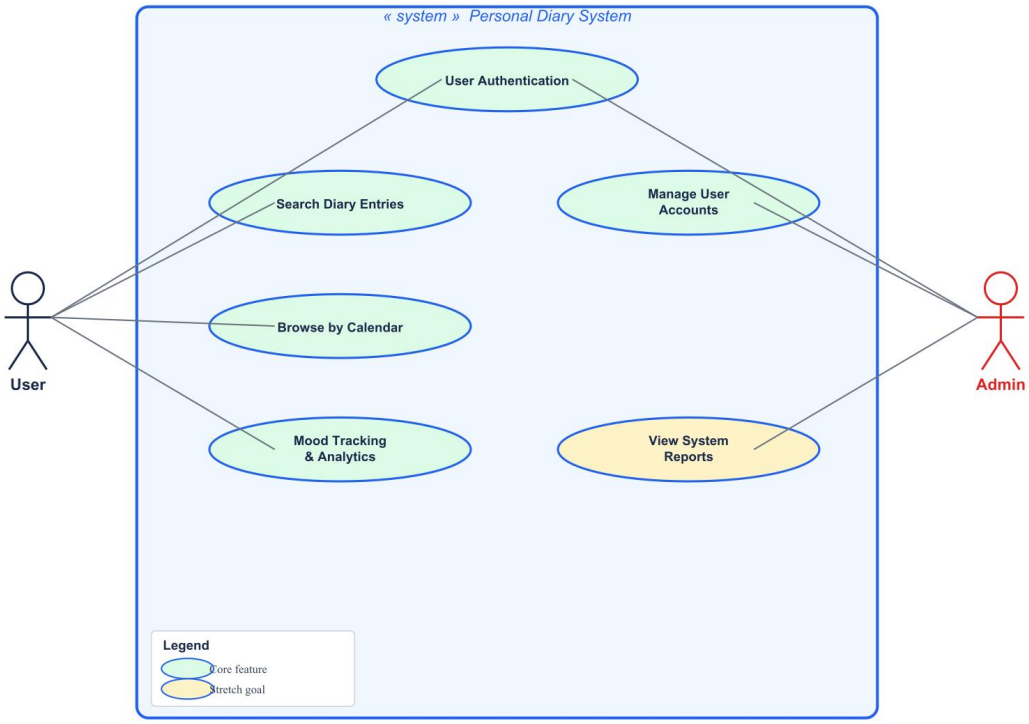

Figure 1 – Use Case Diagram: Personal Diary System

Figure 1 – Use Case Diagram: Personal Diary System

Actors

- User — the diary owner who registers, logs in, writes entries, searches, navigates by calendar, and tracks mood.
- Admin — manages user accounts and also uses User Authentication.

Use Cases

ID	Name	Actor	Priority
UC-01	User Authentication	User + Admin	**Core** ✓
UC-02	Search Diary Entries	User	**Core** ✓
UC-03	Browse by Calendar	User	**Core** ✓
UC-04	Mood Tracking & Analytics	User	**Core** ✓
UC-07	Manage User Accounts	Admin	**Core** ✓

6. Sequence Diagram – User Authentication (FR-01)

The diagram below details the interactions during User Authentication — selected as the main use case because it is a prerequisite for every other function. An alt fragment shows valid vs. invalid credential scenarios.

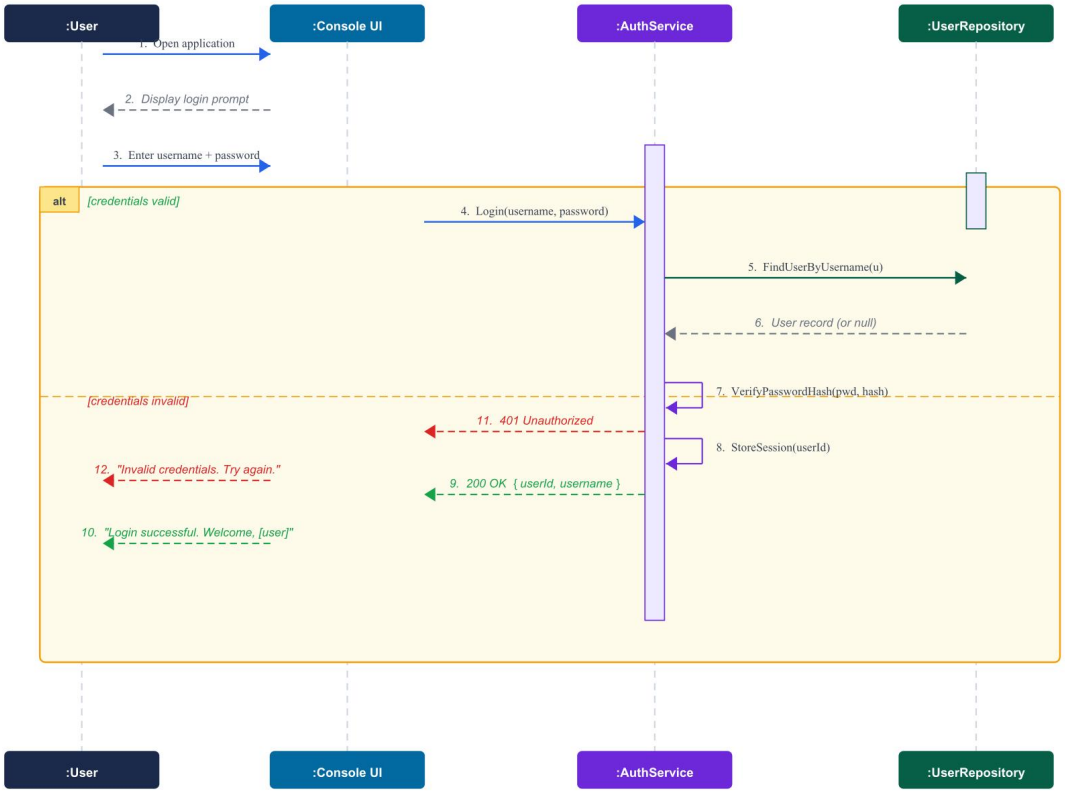

Figure 2 – Sequence Diagram: User Authentication (Main Use Case – FR-01)

Figure 2 – Sequence Diagram: User Authentication

Participants

- :User — the person typing into the console.
- :Console UI — the C# Program.cs menu loop handling all input and output.
- :AuthService — the C# class containing login logic and bcrypt verification.
- :UserRepository — the ADO.NET data access class querying the Users table in SQL Server.

Message Flow

1. User opens application — console displays login prompt.
2. User enters username and password.
3. Console UI calls AuthService.Login(username, password).
4. AuthService calls UserRepository.FindUserByUsername(u).
5. UserRepository executes SELECT query via SqlCommand, returns User record.
6. AuthService calls VerifyPasswordHash(pwd, hash).

[Credentials Valid]

7. AuthService stores userId in session.
8. Returns 200 OK { userId, username } to Console UI.
9. Console UI prints: "Login successful. Welcome, [username]."
10. Main menu is displayed.

[Credentials Invalid]

11. AuthService returns 401 Unauthorized to Console UI.
12. Console UI prints: "Invalid credentials. Try again."